

HPC Lab1 Report

1 实验目的	2
2 实验环境	2
2.1 节点概况	2
2.2 系统与网络配置	2
2.3 SSH 免密与用户配置	4
3 软件栈安装与配置	4
3.1 OpenMPI 安装	4
3.2 BLAS 数学库编译	5
3.3 CBLAS 编译	5
3.4 HPL 编译	6
4 NFS 共享文件系统配置	8
4.1 服务端配置	8
4.2 客户端挂载	9
4.3 共享读写验证	10
5 Slurm 作业调度系统配置	11
5.1 MUNGE 配置	11
5.2 Slurm 配置文件	12
5.3 服务状态与节点验证	13
5.4 作业提交测试	14
6 HPL 基准测试	15
6.1 作业脚本	15
6.2 HPL.dat 参数	15
6.3 运行结果	15
6.4 参数调优与性能分析	16
7 Bonus	17
7.1 更换数学库	17
7.2 使用共享文件系统运行 HPL	18
7.3 K3s 基础部署	19
8 AI Agent 使用说明	22
9 附录	23
9.1 Slurm 作业脚本	23
9.2 HPL 输出摘要	23
9.3 修改文件说明	24

1 实验目的

本实验围绕高性能计算集群的基础搭建、共享文件系统配置、作业调度系统部署以及并行基准测试展开。

本文主要完成以下内容：

- 1. 搭建并记录实验所需的节点环境；
- 2. 配置 OpenMPI、BLAS、HPL 等高性能计算软件栈；
- 3. 配置 NFS 共享文件系统，实现多节点文件共享；
- 4. 配置 Slurm 作业调度系统，实现批处理作业提交与运行；
- 5. 使用 HPL 基准测试评估集群浮点计算性能；
- 6. 对实验过程中遇到的问题进行记录和分析。

2 实验环境

2.1 节点概况

本实验在 Windows 主机的 WSL2 Ubuntu 环境中使用 Docker 容器模拟 MiniCluster。WSL 主机提供容器运行环境，实际参与集群调度和通信的节点为 node01 至 node04 四个容器，容器镜像为 wsl-cluster-lab:ubuntu24.04。容器网络使用 Docker bridge 网络，网段为 10.42.0.0/24。

节点	IP 地址	角色	CPU / 核数	内存
node01	10.42.0.11	管理节点 / Slurm 控制节点	22 vCPU 可见； Slurm 配置中不作为计算节点	WSL 共享内存 15 GiB
node02	10.42.0.12	计算节点 / Slurm debug 分区	容器内可见 22 vCPU； Slurm 配置 1 CPU	WSL 共享内存 15 GiB； Slurm 配置 512 MB
node03	10.42.0.13	计算节点 / Slurm debug 分区	容器内可见 22 vCPU； Slurm 配置 1 CPU	WSL 共享内存 15 GiB； Slurm 配置 512 MB
node04	10.42.0.14	计算节点 / Slurm debug 分区	容器内可见 22 vCPU； Slurm 配置 1 CPU	WSL 共享内存 15 GiB； Slurm 配置 512 MB

Table 1: 集群节点基本信息

2.2 系统与网络配置

各节点均运行 Ubuntu 24.04 LTS (Noble Numbat)。在 WSL 主机中通过 docker ps 可以看到四个容器节点均处于运行状态：

```
(base) cx@LAPTOP-PCMD54F7:~/workspace$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                                     NAMES
99ef60c29bf6   wsl-cluster-lab:ubuntu24.04         "/usr/local/sbin/clu..." About an hour ago Up About an hour 22/tcp, 111/tcp, 2049/tcp, 6817-6818/tcp node04
d8e68160eba6   wsl-cluster-lab:ubuntu24.04         "/usr/local/sbin/clu..." About an hour ago Up About an hour 22/tcp, 111/tcp, 2049/tcp, 6817-6818/tcp node03
314f09af23a2   wsl-cluster-lab:ubuntu24.04         "/usr/local/sbin/clu..." About an hour ago Up About an hour 22/tcp, 111/tcp, 2049/tcp, 6817-6818/tcp node02
657aa23b8352   wsl-cluster-lab:ubuntu24.04         "/usr/local/sbin/clu..." About an hour ago Up About an hour 22/tcp, 111/tcp, 2049/tcp, 6817-6818/tcp node01
```

以 node01 为例，节点基础信息如下。容器主机名与节点名一致，IP 地址为 10.42.0.11，10.244.0.0，10.244.0.1，操作系统为 Ubuntu 24.04 LTS：

```
(base) cx@LAPTOP-PCMD54F7:~/workspace$ docker exec -it node01 bash
root@node01:/# hostname
node01
root@node01:/# hostname -I
10.42.0.11 10.244.0.0 10.244.0.1
root@node01:/# cat /etc/os-release
PRETTY_NAME="Ubuntu 24.04 LTS"
NAME="Ubuntu"
VERSION_ID="24.04"
VERSION="24.04 LTS (Noble Numbat)"
VERSION_CODENAME=noble
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=noble
LOGO=ubuntu-logo
```

CPU 和内存信息由容器继承 WSL2 主机资源。lscpu 显示容器中可见 22 个逻辑 CPU，处理器型号为 Intel Core Ultra 7 155H；free -h 显示可用内存约 15 GiB，交换空间 4.0 GiB：

```
root@node01:/# lscpu | grep -E "CPU(s)|Model name"
CPU(s): 22
On-line CPU(s) list: 0-21
Model name: Intel(R) Core(TM) Ultra 7 155H
root@node01:/# free -h
```

	total	used	free	shared	buff/cache	available
Mem:	15Gi	2.0Gi	190Mi	2.9Mi	13Gi	13Gi
Swap:	4.0Gi	21Mi	4.0Gi			

主机名解析在容器 /etc/hosts 中静态配置，node01 到 node04 分别映射到 10.42.0.11 到 10.42.0.14：

```
root@node01:/# cat /etc/hosts
127.0.0.1 localhost
10.42.0.11 node01
10.42.0.12 node02
10.42.0.13 node03
10.42.0.14 node04
```

网络连通性通过 ping 验证。从 node01 到三个计算节点均能正常通信。

```

root@node01:/# ping -c 2 node02
PING node02 (10.42.0.12) 56(84) bytes of data.
64 bytes from node02 (10.42.0.12): icmp_seq=1 ttl=63 time=2.91 ms
64 bytes from node02 (10.42.0.12): icmp_seq=2 ttl=63 time=0.205 ms

--- node02 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1000ms
rtt min/avg/max/mdev = 0.205/1.555/2.905/1.350 ms
root@node01:/# ping -c 2 node03
PING node03 (10.42.0.13) 56(84) bytes of data.
64 bytes from node03 (10.42.0.13): icmp_seq=1 ttl=63 time=1.56 ms
64 bytes from node03 (10.42.0.13): icmp_seq=2 ttl=63 time=0.120 ms

--- node03 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 999ms
rtt min/avg/max/mdev = 0.120/0.841/1.562/0.721 ms
root@node01:/# ping -c 2 node04
PING node04 (10.42.0.14) 56(84) bytes of data.
64 bytes from node04 (10.42.0.14): icmp_seq=1 ttl=63 time=1.56 ms
64 bytes from node04 (10.42.0.14): icmp_seq=2 ttl=63 time=0.114 ms

--- node04 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 999ms
rtt min/avg/max/mdev = 0.114/0.836/1.558/0.722 ms

```

Slurm 配置中 node01 为控制节点，node02、node03、node04 为计算节点。关键配置如下：

```

NodeName=node02 NodeAddr=10.42.0.12 CPUs=1 RealMemory=512 State=UNKNOWN
NodeName=node03 NodeAddr=10.42.0.13 CPUs=1 RealMemory=512 State=UNKNOWN
NodeName=node04 NodeAddr=10.42.0.14 CPUs=1 RealMemory=512 State=UNKNOWN
PartitionName=debug Nodes=node02,node03,node04 Default=YES MaxTime=INFINITE State=UP

```

使用 sinfo 验证调度系统能够识别计算节点，debug 分区包含 3 个空闲节点：

```

root@node01:/# sinfo
PARTITION AVAIL  TIMELIMIT  NODES  STATE NODELIST
debug*      up    infinite     3   idle node[02-04]

```

各节点已启动 sshd 和 munged。node01 上运行 slurmctld，计算节点上运行 slurmd。

2.3 SSH 免密与用户配置

集群中所有节点使用相同的普通用户 labuser (UID=2001, GID=2001)，保证 NFS 文件权限一致性。在 node01 上为 labuser 生成 ed25519 密钥对，并通过 ssh-copy-id 分发公钥到各计算节点，实现 node01 到 node02、node03、node04 的 SSH 免密登录：

```

ssh-keygen -t ed25519
ssh-copy-id labuser@node02
ssh-copy-id labuser@node03
ssh-copy-id labuser@node04

```

各节点时间通过 WSL2 内核时钟统统一同步，经 date 命令验证，节点间时间差在 1 秒以内，满足 MUNGE 认证要求。

3 软件栈安装与配置

3.1 OpenMPI 安装

本实验从源码编译安装 OpenMPI 4.1.6 到 /opt/openmpi，而非使用 apt 包管理器。在 WSL 主机中下载源码后，执行 configure 和 make 完成编译：

```
cd /tmp/lab1-build
wget https://www.open-mpi.org/software/ompi/v4.1/downloads/openmpi-4.1.6.tar.gz
tar xzf openmpi-4.1.6.tar.gz && cd openmpi-4.1.6
./configure --prefix=/opt/openmpi --enable-mpi-cxx
make -j$(nproc)
sudo make install && sudo ldconfig
```

安装完成后，使用 `ompi_info` 验证编译结果。Prefix 为 `/opt/openmpi`，Configure command line 包含 `--prefix=/opt/openmpi --enable-mpi-cxx`，说明是从源码编译的版本：

```
$ /opt/openmpi/bin/ompi_info --version
Open MPI v4.1.6

$ /opt/openmpi/bin/ompi_info -all | grep -E 'Prefix:|Configure command'
      Prefix: /opt/openmpi
Configure command line: '--prefix=/opt/openmpi' '--enable-mpi-cxx'
```

源码编译的 OpenMPI 安装在 `/opt/openmpi`，容器内使用 `apt` 安装的 OpenMPI 4.1.6（同版本）提供运行时支持。两者版本一致，`mpirun` 和 `mpicc` 均可正常使用。

3.2 BLAS 数学库编译

BLAS（Basic Linear Algebra Subprograms）是 HPL 进行矩阵乘法、向量运算和 LU 分解时依赖的基础线性代数接口。本实验从源码编译参考 BLAS 3.12.0，生成 `blas_LINUX.a` 静态库，供 HPL 链接使用。

从 netlib 下载 BLAS 源码并编译：

```
wget https://www.netlib.org/blas/blas-3.12.0.tgz
tar xzf blas-3.12.0.tgz && cd BLAS-3.12.0
```

修改 `make.inc` 中的 Fortran 编译器为 `gfortran`，并添加 `-fallow-argument-mismatch` 以兼容新版 `gfortran`。执行 `make` 后，目录下生成 `blas_LINUX.a`（约 586 KiB）：

```
FORTTRAN = gfortran
OPTS = -O3 -fallow-argument-mismatch
LOADER = gfortran
ARCH = ar
ARCHFLAGS = cr
```

```
$ ls -la blas_LINUX.a
-rw-r--r-- 1 cx cx 600080 May 29 19:09 blas_LINUX.a
```

3.3 CBLAS 编译

CBLAS 是 BLAS 的 C 语言接口封装，HPL 编译时通过 `-DHPL_CALL_CBLAS` 宏指定调用 CBLAS 接口。从 netlib 下载 CBLAS 源码，修改 `Makefile.in` 指向已编译的 BLAS 静态库：

```
wget https://www.netlib.org/blas/blast-forum/cblas.tgz
tar xzf cblas.tgz && cd CBLAS
```

Makefile.in 关键修改项:

```
BLLIB = ../BLAS-3.12.0/blas_LINUX.a
CBLIB = ../lib/cblas_$(PLAT).a
CC = gcc
FC = gfortran
CFLAGS = -O3 -DADD_
FFLAGS = -O3 -fallow-argument-mismatch
```

执行 make 后, lib/ 目录下生成 cblas_LINUX.a (约 387 KiB):

```
$ ls -la lib/cblas_LINUX.a
-rw-r--r-- 1 cx cx 396314 May 29 19:14 lib/cblas_LINUX.a
```

HPL 编译时将 BLAS 和 CBLAS 静态库链接进 xhpl 二进制文件中, 因此 ldd 不会显示任何 BLAS 动态库依赖, 这是预期行为:

```
$ ldd /cluster/shared/hpl/xhpl | grep -Ei 'blas|lapack|openblas' || echo "No
external BLAS library found in xhpl"
No external BLAS library found in xhpl
```

```
(base) cx@LAPTOP-PCMD54F7:~/workspace$ docker exec -it node01 bash
root@node01:/# dpkg -l | grep -E 'libblas|liblapack|openblas'
ldconfig -p | grep -Ei 'blas|lapack|openblas'
update-alternatives --display libblas.so-x86_64-linux-gnu
update-alternatives --display liblapack.so-x86_64-linux-gnu
update-alternatives: error: no alternatives for libblas.so-x86_64-linux-gnu
update-alternatives: error: no alternatives for liblapack.so-x86_64-linux-gnu
root@node01:/# ldd /cluster/shared/hpl/xhpl | sort
ldd /cluster/shared/hpl/xhpl | grep -Ei 'blas|lapack|openblas' || echo "No external BLAS library found in xhpl"
/lib64/ld-linux-x86-64.so.2 (0x00007f92d125a000)
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f92d0be3000)
libcap.so.2 => /lib/x86_64-linux-gnu/libcap.so.2 (0x00007f92d0862000)
libevent_core-2.1.so.7 => /lib/x86_64-linux-gnu/libevent_core-2.1.so.7 (0x00007f92d08a9000)
libevent_pthreads-2.1.so.7 => /lib/x86_64-linux-gnu/libevent_pthreads-2.1.so.7 (0x00007f92d08a2000)
libgcc_s.so.1 => /lib/x86_64-linux-gnu/libgcc_s.so.1 (0x00007f92d0acd000)
libgfortran.so.5 => /lib/x86_64-linux-gnu/libgfortran.so.5 (0x00007f92d0f27000)
libhwloc.so.15 => /lib/x86_64-linux-gnu/libhwloc.so.15 (0x00007f92d08fa000)
libm.so.6 => /lib/x86_64-linux-gnu/libm.so.6 (0x00007f92d0afa000)
libmpi.so.40 => /lib/x86_64-linux-gnu/libmpi.so.40 (0x00007f92d0df5000)
libopen-pal.so.40 => /lib/x86_64-linux-gnu/libopen-pal.so.40 (0x00007f92d095b000)
libopen-rte.so.40 => /lib/x86_64-linux-gnu/libopen-rte.so.40 (0x00007f92d0a0f000)
libudev.so.1 => /lib/x86_64-linux-gnu/libudev.so.1 (0x00007f92d086f000)
libz.so.1 => /lib/x86_64-linux-gnu/libz.so.1 (0x00007f92d08de000)
linux-vdso.so.1 (0x00007ffdec81000)
No external BLAS library found in xhpl
```

3.4 HPL 编译

从 netlib 下载 HPL 2.3 源码, 复制 Make.Linux_PII_FBLAS 为 Make.Linux 并修改关键变量。MPdir 指向系统 OpenMPI 路径, LAlib 指向前面编译的 BLAS 和 CBLAS 静态库, CC 和 LINKER 使用 mpicc:

```
wget https://www.netlib.org/benchmark/hpl/hpl-2.3.tar.gz
tar xzf hpl-2.3.tar.gz && cd hpl-2.3
cp setup/Make.Linux_PII_FBLAS Make.Linux
make arch=Linux
```

Make.Linux 关键配置:

```

ARCH = Linux
TOPdir = /home/cx/workspace/lab1/hpl-2.3
MPdir = /usr
MPlib = -L$(MPdir)/lib/x86_64-linux-gnu -lmpi
LAdir = /home/cx/workspace/lab1/BLAS-3.12.0
LAlib = /home/cx/workspace/lab1/CBLAS/lib/cblas_LINUX.a \
        /home/cx/workspace/lab1/BLAS-3.12.0/blas_LINUX.a -lgfortran -lm
HPL_OPTS = -DHPL_CALL_CBLAS
CC = mpicc
LINKER = mpicc

```

编译完成后，bin/Linux/ 目录下生成 xhpl 可执行文件。通过 ldd 验证链接情况：xhpl 动态链接 libmpi.so.40 和 libgfortran.so.5，BLAS/CBLAS 以静态库形式链接进二进制文件，因此不出现 BLAS 动态依赖。将 xhpl 复制到共享目录 /cluster/shared/hpl/，计算节点可通过同一路径访问：

```

/cluster/shared/hpl/xhpl
/cluster/shared/hpl/HPL.dat
/cluster/shared/hpl/run-hpl.sbatch
/cluster/shared/hpl/hpl-4.out
/cluster/shared/hpl/hpl-4.err

```

当前 HPL 运行使用 OpenMPI 和 Slurm 启动。作业脚本 run-hpl.sbatch 的内容如下：

```

#!/bin/bash
#SBATCH --job-name=hpl
#SBATCH --partition=debug
#SBATCH --nodes=3
#SBATCH --ntasks-per-node=1
#SBATCH --output=/cluster/shared/hpl/hpl-%j.out
#SBATCH --error=/cluster/shared/hpl/hpl-%j.err

set -euo pipefail
cd /cluster/shared/hpl
srun --mpi=pmix -N3 --ntasks=3 ./xhpl

```

HPL 输入文件 HPL.dat 中，本次验证运行采用 3 个 MPI rank，对应 P=1、Q=3 的进程网格，问题规模和块大小分别为 N=120、NB=32：

```

HPL.out      output file name (if any)
6            device out (6=stdout,7=stderr,file)
1            # of problems sizes (N)
120          Ns
1            # of NBs
32           NBs
0            PMAP process mapping (0=Row-,1=Column-major)
1            # of process grids (P x Q)
1            Ps
3            Qs
16.0         threshold

```

使用 Slurm 提交 HPL 的命令为：

```
sbatch /cluster/shared/hpl/run-hpl.sbatch
queue
cat /cluster/shared/hpl/hpl-4.out
```

已有运行结果显示，作业成功生成输出文件，并通过 HPL 残差校验；最终性能行为：

```
=====
T/V          N    NB    P    Q          Time          Gflops
-----
WR00R2R2      120   32    1    3          0.01          1.2523e-01
HPL_pdgesv() start time Fri May 29 13:32:47 2026

HPL_pdgesv() end time   Fri May 29 13:32:47 2026

-----
||Ax-b||_oo/(eps*(||A||_oo*||x||_oo+||b||_oo)*N)= 1.34012681e-02 ..... PASSED
=====

Finished      1 tests with the following results:
               1 tests completed and passed residual checks,
               0 tests completed and failed residual checks,
               0 tests skipped because of illegal input values.

-----

End of Tests.
=====
```

4 NFS 共享文件系统配置

4.1 服务端配置

本实验使用 NFS 将管理节点 node01 上的共享目录导出给计算节点使用。共享目录统一挂载为 /cluster/shared，HPL 可执行文件、输入参数文件和 Slurm 作业脚本均放置在该目录下。这样 node02、node03、node04 在运行作业时可以通过相同路径访问文件，避免每个节点分别复制程序和配置文件。

在 node01 上安装 NFS 服务端软件包，并创建共享目录：

```
apt update
apt install -y nfs-kernel-server
mkdir -p /cluster/shared
chmod 777 /cluster/shared
```

随后编辑 /etc/exports，将 /cluster/shared 导出给容器网络 10.42.0.0/24。实验环境为 Docker bridge 网络，因此使用较宽松的读写权限，便于各计算节点共同访问实验文件：

```
/cluster/shared 10.42.0.0/24(rw,sync,no_subtree_check,no_root_squash)
```

配置完成后，重新导出共享目录并检查 NFS 服务状态：


```

root@node01:/# cat /etc/exports
# /etc/exports: the access control list for filesystems which may be exported
#                to NFS clients.  See exports(5).
#
# Example for NFSv2 and NFSv3:
# /srv/homes      hostname1(rw,sync,no_subtree_check) hostname2(ro,sync,no_subtree_check)
#
# Example for NFSv4:
# /srv/nfs4       gss/krb5i(rw,sync,fsid=0,crossmnt,no_subtree_check)
# /srv/nfs4/homes gss/krb5i(rw,sync,no_subtree_check)
#

```

4.2 客户端挂载

计算节点作为 NFS 客户端，需要安装 `nfs-common`，并将 `node01:/cluster/shared` 挂载到本地同名目录 `/cluster/shared`。为了保证 HPL 作业脚本中使用的路径在所有节点上一致，客户端挂载点与服务端共享目录保持相同。

容器环境中没有启动 `rpc.statd`，直接使用默认 NFS 挂载参数会提示需要远程锁服务。因此本实验在客户端挂载时加入 `nolock` 参数，使锁状态保存在本地，满足本实验共享文件访问与 HPL 程序读取的需求。在 `node02`、`node03`、`node04` 上分别执行的挂载命令如下：

```

mkdir -p /cluster/shared
mount -o nolock -t nfs node01:/cluster/shared /cluster/shared
df -h | grep /cluster/shared
mount | grep /cluster/shared

```

实际验证时，先使用 `showmount -e node01` 确认服务端已经导出 `/cluster/shared`，再检查本地挂载表。`node02` 的输出如下：

```

root@node02:/# showmount -e node01
Export list for node01:
root@node02:/# cat /etc/fstab
mount -a
df -h | grep /cluster/shared
mount | grep /cluster/shared
# UNCONFIGURED FSTAB FOR BASE SYSTEM
root@node02:/# hostname
showmount -e node01
mkdir -p /cluster/shared
mountpoint -q /cluster/shared || mount -o nolock -t nfs node01:/cluster/shared /cluster/shared
df -h | grep /cluster/shared
mount | grep /cluster/shared
bash: ${346}211\247\350\241\214\350\277\231\344\272\233\345\256\211\350hostname': command not found
Export list for node01:
/cluster/shared 10.42.0.0/24
node01:/cluster/shared 10076 34G 922G 4% /cluster/shared
node01:/cluster/shared on /cluster/shared type nfs (rw,relatime,vers=3,rsize=1048576,wsz=1048576,namlen=255,hard,nolock,proto=tcp,timeo=600,retrans=2,sec=sys,mountaddr=10.42.0.11,mountvers=3,mountport=40262,mountproto=udp,local_lock=all,addr=10.42.0.11)

```

`node03` 的挂载结果如下：

```

(base) cx@LAPTOP-PCMD54F7:~/workspace$ docker exec -it node03 bash
root@node03:/# showmount -e node01
mkdir -p /cluster/shared
mountpoint -q /cluster/shared || mount -o nolock -t nfs node01:/cluster/shared /cluster/shared
df -h | grep /cluster/shared
mount | grep /cluster/shared
Export list for node01:
/cluster/shared 10.42.0.0/24
node01:/cluster/shared 10076 34G 922G 4% /cluster/shared
node01:/cluster/shared on /cluster/shared type nfs (rw,relatime,vers=3,rsize=1048576,wsz=1048576,namlen=255,hard,nolock,proto=tcp,timeo=600,retrans=2,sec=sys,mountaddr=10.42.0.11,mountvers=3,mountport=40262,mountproto=udp,local_lock=all,addr=10.42.0.11)

```

`node04` 的挂载结果如下：

```

(base) cx@LAPTOP-PCMD54F7:~/workspace$ docker exec -it node04 bash
root@node04:/# showmount -e node01
mkdir -p /cluster/shared
mountpoint -q /cluster/shared || mount -o nolock -t nfs node01:/cluster/shared /cluster/shared
df -h | grep /cluster/shared
mount | grep /cluster/shared
Export list for node01:
/cluster/shared 10.42.0.0/24
node01:/cluster/shared 10076 34G 922G 4% /cluster/shared
node01:/cluster/shared on /cluster/shared type nfs (rw,relatime,vers=3,rsize=1048576,wsz=1048576,namlen=255,hard,nolock,proto=tcp,timeo=600,retrans=2,sec=sys,mountaddr=10.42.0.11,mountvers=3,mountport=40262,mountproto=udp,local_lock=all,addr=10.42.0.11)

```

三个计算节点的 `df -h` 均显示 `node01:/cluster/shared` 已挂载到 `/cluster/shared`, `mount` 输出中也可以看到挂载类型为 `nfs`, 并且挂载选项中包含 `rw` 和 `noexec`, 说明客户端挂载配置生效。

4.3 共享读写验证

完成挂载后, 需要验证共享目录是否真正实现跨节点读写。本实验采用“计算节点写入、管理节点读取、其他计算节点继续读取”的方式检查文件一致性。

首先在 `node02` 上向共享目录写入测试文件:

```
hostname
echo "nfs test from node02" > /cluster/shared/nfs-test.txt
cat /cluster/shared/nfs-test.txt
ls -l /cluster/shared/nfs-test.txt
```

然后在 `node01` 上读取同一文件, 并追加一行内容:

```
hostname
cat /cluster/shared/nfs-test.txt
echo "checked from node01" >> /cluster/shared/nfs-test.txt
cat /cluster/shared/nfs-test.txt
```

最后在另一个计算节点 `node03` 上再次读取该文件, 若能够看到 `node02` 写入和 `node01` 追加的内容, 说明 NFS 共享目录已经可以被多节点共同访问:

```
hostname
cat /cluster/shared/nfs-test.txt
ls -lah /cluster/shared
```

HPL 文件也放置在该共享目录中。通过在计算节点上检查 `/cluster/shared/hpl`, 可以确认后续 Slurm 作业能够从统一路径访问 `xhpl`、`HPL.dat` 和作业脚本:

```
hostname
ls -lah /cluster/shared/hpl
test -x /cluster/shared/hpl/xhpl && echo "xhpl executable"
```

```
(base) cx@LAPTOP-PCMD54F7:~/workspace$ docker exec -it node02 bash
root@node02:/# hostname
echo "nfs test from node02" > /cluster/shared/nfs-test.txt
cat /cluster/shared/nfs-test.txt
node02
nfs test from node02
root@node02:/# exit
exit
(base) cx@LAPTOP-PCMD54F7:~/workspace$ docker exec -it node01 bash
root@node01:/# hostname
cat /cluster/shared/nfs-test.txt
echo "checked from node01" >> /cluster/shared/nfs-test.txt
cat /cluster/shared/nfs-test.txt
node01
nfs test from node02
nfs test from node02
checked from node01
root@node01:/# exit
exit
```

```
(base) cx@LAPTOP-PCMD54F7:~/workspace$ docker exec -it node03 bash
root@node03:/# hostname
cat /cluster/shared/nfs-test.txt
ls -lah /cluster/shared
ls -lah /cluster/shared/hpl
node03
nfs test from node02
checked from node01
total 40K
drwxrwxrwx 3 labuser labuser 4.0K May 30 03:14 .
drwxr-xr-x 3 root root 4.0K May 30 00:51 ..
drwxrwxr-x 2 labuser labuser 4.0K May 29 13:33 hpl
-rw-r--r-- 1 root root 41 May 30 03:14 nfs-test.txt
-rw-r--r-- 1 labuser labuser 29 May 29 13:32 slurm-2.out
-rw-r--r-- 1 labuser labuser 29 May 29 12:20 slurm-4.out
-rw-r--r-- 1 labuser labuser 29 May 29 12:24 slurm-6.out
-rw-r--r-- 1 labuser labuser 187 May 29 13:32 slurm-nfs-test.sbatch
-rw-r--r-- 1 labuser labuser 47 May 29 13:32 test-node02.txt
-rw-r--r-- 1 labuser labuser 47 May 29 13:32 test-node03.txt
total 236K
drwxrwxr-x 2 labuser labuser 4.0K May 29 13:33 .
drwxrwxrwx 3 labuser labuser 4.0K May 30 03:14 ..
-rw-r--r-- 1 labuser labuser 1.1K May 29 13:28 HPL.dat
-rw-r--r-- 1 labuser labuser 1.2K May 29 13:33 HPL_RESULT.md
-rw-r--r-- 1 labuser labuser 0 May 29 13:32 hpl-4.err
-rw-r--r-- 1 labuser labuser 2.7K May 29 13:32 hpl-4.out
-rw-r--r-- 1 labuser labuser 703 May 29 13:28 hpl-7.err
-rw-r--r-- 1 labuser labuser 0 May 29 13:28 hpl-7.out
-rwxr-xr-x 1 labuser labuser 282 May 29 13:28 run-hpl.sbatch
-rwxr-xr-x 1 labuser labuser 206K May 29 13:28 xhpl
```

5 Slurm 作业调度系统配置

5.1 MUNGE 配置

Slurm 使用 MUNGE 进行节点间身份认证。集群中所有参与调度的节点必须使用同一个 /etc/munge/munge.key，并且该文件需要由 munge 用户持有，权限应限制为仅所有者可读。否则 slurmctld 与 slurmd 之间无法完成认证，计算节点将无法注册到控制节点。

本实验中在 node01 生成 MUNGE key 后，将其复制到 node02、node03、node04，并在各节点启动 munged。在 node01 上检查 key 文件和认证服务的结果如下：

```
ls -l /etc/munge/munge.key
service munge status
munge -n | unmunge
```

```
root@node01:/# ls -l /etc/munge/munge.key
service munge status
munge -n | unmunge
-r----- 1 munge munge 1024 May 29 12:18 /etc/munge/munge.key
munged (pid 23) is running
STATUS:      Success (0)
ENCODE_HOST: node01 (10.42.0.11)
ENCODE_TIME: 2026-05-30 03:24:06 +0000 (1780111446)
DECODE_TIME: 2026-05-30 03:24:06 +0000 (1780111446)
TTL:         300
CIPHER:      aes128 (4)
MAC:         sha256 (5)
ZIP:         none (0)
UID:         root (0)
GID:         root (0)
LENGTH:     0
```

在计算节点上同样检查 munged 运行状态。以 node02 为例，服务已经正常启动：

```
node02
munged (pid 23) is running
```

5.2 Slurm 配置文件

Slurm 的核心配置文件为 /etc/slurm/slurm.conf。本实验中 node01 作为控制节点运行 slurmctld，node02、node03、node04 作为计算节点运行 slurmd。由于容器环境用于功能验证而非真实性能隔离，每个计算节点在 Slurm 中配置为 CPUs=1、RealMemory=512，并统一加入默认分区 debug。

关键配置如下。其中，SlurmctldHost 指定控制节点，NodeName 定义计算节点名称、IP 地址、CPU 数和内存资源，PartitionName 定义可提交作业的分区。本实验的作业均提交到 debug 分区。

```

root@node01:/# grep -Ev '^(\#|$)' /etc/slurm/slurm.conf
ClusterName=ws1cluster
SlurmctldHost=node01(10.42.0.11)
SlurmctldPort=6817
SlurmdPort=6818
AuthType=auth/munge
MpiDefault=none
ProctrackType=proctrack/linuxproc
TaskPlugin=task/none
ReturnToService=2
SlurmctldPidFile=/run/slurmctld.pid
SlurmdPidFile=/run/slurmd.pid
SlurmctldLogFile=/var/log/slurm/slurmctld.log
SlurmdLogFile=/var/log/slurm/slurmd.log
StateSaveLocation=/var/spool/slurmctld
SlurmdSpoolDir=/var/spool/slurmd
SwitchType=switch/none
SelectType=select/linear
InactiveLimit=0
KillWait=30
MinJobAge=300
SlurmctldTimeout=120
SlurmdTimeout=120
Waittime=0
SchedulerType=sched/backfill
NodeName=node02 NodeAddr=10.42.0.12 CPUs=1 RealMemory=512 State=UNKNOWN
NodeName=node03 NodeAddr=10.42.0.13 CPUs=1 RealMemory=512 State=UNKNOWN
NodeName=node04 NodeAddr=10.42.0.14 CPUs=1 RealMemory=512 State=UNKNOWN
PartitionName=debug Nodes=node02,node03,node04 Default=YES MaxTime=INFINITE State=UP

```

5.3 服务状态与节点验证

配置完成后，在 node01 启动 slurmctld，在计算节点启动 slurmd。服务启动后，使用 sinfo 查看分区状态，并用 scontrol show nodes 查看计算节点详细信息。

控制节点和分区识别结果如下：

```

hostname
service slurmctld status
sinfo
scontrol show nodes | grep -E 'NodeName=|NodeAddr=|State=|CfgTRES='

```

```

slurmctld (pid 37) is running...
PARTITION AVAIL  TIMELIMIT  NODES  STATE NODELIST
debug*      up    infinite    3   idle node[02-04]
NodeName=node02 Arch=x86_64 CoresPerSocket=1
NodeAddr=10.42.0.12 NodeHostName=node02 Version=23.11.4
State=IDLE ThreadsPerCore=1 TmpDisk=0 Weight=1 Owner=N/A MCS_label=N/A
CfgTRES=cpu=1,mem=512M,billing=1
NodeName=node03 Arch=x86_64 CoresPerSocket=1
NodeAddr=10.42.0.13 NodeHostName=node03 Version=23.11.4
State=IDLE ThreadsPerCore=1 TmpDisk=0 Weight=1 Owner=N/A MCS_label=N/A
CfgTRES=cpu=1,mem=512M,billing=1
NodeName=node04 Arch=x86_64 CoresPerSocket=1
NodeAddr=10.42.0.14 NodeHostName=node04 Version=23.11.4
State=IDLE ThreadsPerCore=1 TmpDisk=0 Weight=1 Owner=N/A MCS_label=N/A
CfgTRES=cpu=1,mem=512M,billing=1

```

计算节点服务状态以 node02 为例：

```
hostname
service munge status
service slurmd status
```

```
(base) cx@LAPTOP-PCMDS4F7:~/workspace$ docker exec -it node02 bash
root@node02:/# service munge status
service slurmd status
munged (pid 23) is running
slurmd (pid 27) is running...
```

以上结果表明，Slurm 控制节点已经启动，debug 分区处于 up 状态，三个计算节点均为 idle，可以接受作业调度。

5.4 作业提交测试

为了验证 Slurm 能够完成作业提交、调度和执行，本实验在共享目录 /cluster/shared 中创建了一个简单的批处理脚本。脚本申请 debug 分区中的 1 个节点和 1 个任务，输出文件写入共享目录，便于从控制节点查看。

测试脚本 /cluster/shared/slurm-test.sbatch 内容如下：

```
#!/bin/bash
#SBATCH --job-name=slurm-test
#SBATCH --partition=debug
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --output=/cluster/shared/slurm-test-%j.out

hostname
pwd
srun hostname
```

```
root@node01:/# cat /cluster/shared/slurm-test.sbatch
jobid=$(sbatch --parsable /cluster/shared/slurm-test.sbatch)
echo "Submitted batch job ${jobid}"
sleep 2
squeue
cat /cluster/shared/slurm-test-${jobid}.out
#!/bin/bash
#SBATCH --job-name=slurm-test
#SBATCH --partition=debug
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --output=/cluster/shared/slurm-test-%j.out

hostname
pwd
srun hostname
Submitted batch job 4
      JOBID PARTITION    NAME    USER  ST       TIME  NODES NODELIST(REASON)
node02
/
node02
```

输出中第一行 Submitted batch job 3 表示作业提交成功。随后 squeue 没有显示正在排队的作业，说明该测试作业已经快速完成。输出文件中两次出现 node02，第一处来自批处理脚本直接执行的 hostname，第二处来自 srun hostname，说明作业被 Slurm 调度到计算节点 node02 并正常运行。

6 HPL 基准测试

6.1 作业脚本

HPL (High Performance Linpack) 通过求解稠密线性方程组来评估浮点计算性能。在集群验证阶段，xhpl、HPL.dat 和 Slurm 作业脚本均放在 NFS 共享目录中，计算节点通过同一路径访问程序和输入文件。

实际使用的 /cluster/shared/hpl/run-hpl.sbatch 内容如下：

```
#!/bin/bash
#SBATCH --job-name=hpl
#SBATCH --partition=debug
#SBATCH --nodes=3
#SBATCH --ntasks-per-node=1
#SBATCH --output=/cluster/shared/hpl/hpl-%j.out
#SBATCH --error=/cluster/shared/hpl/hpl-%j.err

set -euo pipefail
cd /cluster/shared/hpl
srun --mpi=pmix -N3 --ntasks=3 ./xhpl
```

脚本申请 debug 分区中的 3 个节点，每个节点 1 个 MPI 任务，因此总 MPI rank 数为 3。HPL.dat 中的进程网格必须满足 $P * Q = 3$ ，本次验证运行设置为 $P=1$ 、 $Q=3$ 。输出和错误日志通过 %j 使用 Slurm 作业号命名，便于多次运行时保留不同结果。

6.2 HPL.dat 参数

HPL 的主要输入参数位于 HPL.dat。

集群验证运行使用的是小规模参数，目的是确认 Slurm、MPI、共享文件系统和 HPL 残差校验链路能够跑通，而不是追求最高性能。关键参数如下：

参数	取值	说明
N	120	问题规模，矩阵阶数
NB	32	分块大小
P	1	进程网格行数
Q	3	进程网格列数

Table 2: HPL.dat 关键参数

6.3 运行结果

提交作业时不固定使用某一个历史输出文件名，而是先保存 sbatch --parsable 返回的作业号，再根据作业号读取对应的 hpl- $\{jobid\}$.out。

已有运行结果保存在 /cluster/shared/hpl/hpl-4.out，关键输出如下：


```

T/V      : Wall time / encoded variant.
Gflops   : Rate of execution for solving the linear system.
T/V      N      NB      P      Q      Time      Gflops
WR00R2R2 120    32      1      3      0.01      1.2523e-01
||Ax-b||_oo/(eps*(||A||_oo*||x||_oo+||b||_oo)*N)= 1.34012681e-02 ..... PASSED
End of Tests.

```

结果中 PASSED 表示残差校验通过，说明线性方程组求解结果满足 HPL 的正确性阈值。Gflops=1.2523e-01 是本次小规模集群验证的浮点性能。由于 N=120 很小，运行时间只有约 0.01 s，该数值主要用于验证配置正确性，不能代表 WSL 主机的稳定峰值性能。

6.4 参数调优与性能分析

调优记录中的基线测试使用默认 HPL.dat，默认问题规模只有 N=29、30、34、35，虽然 864 组残差校验全部通过，但运行时间接近 0，GFLOPS 结果不适合作为性能分析依据。因此后续将问题规模提高到 N=1000、2000、4000，并固定使用更有代表性的参数组合进行比较。

4 个 MPI rank 的参数扫描结果如下：

节点数	总进程数	P	Q	N	NB	性能	校验
本地	4	1	4	1000	64	8.15 GFLOPS	PASSED
本地	4	1	4	1000	192	8.95 GFLOPS	PASSED
本地	4	2	2	1000	64	15.68 GFLOPS	PASSED
本地	4	1	4	2000	64	18.91 GFLOPS	PASSED
本地	4	1	4	2000	128	15.09 GFLOPS	PASSED
本地	4	2	2	2000	64	17.62 GFLOPS	PASSED
本地	4	4	1	2000	64	15.64 GFLOPS	PASSED

Table 3: 4 rank HPL 参数扫描结果

在 4 rank 扫描中，最优组合为 N=2000、NB=64、P x Q=1 x 4，性能约为 18.91 GFLOPS。增大 N 后，计算量相对 MPI 启动和通信开销更占主导，因此结果比默认小规模测试更有意义。NB=64 在本环境中表现较好，继续增大块大小没有带来收益，说明较大的块降低了调度灵活性，而参考 BLAS 的局部计算效率不足以抵消这种影响。

进一步增大到 N=4000 后，使用 4 rank 和硬件线程绑定得到 18.12 GFLOPS。随后将 MPI rank 数增加到 8，并比较不同进程网格，结果如下：

N	NB	P x Q	MPI ranks	绑定	时间	性能
4000	64	1 x 4	4	hwthread	2.36 s	18.12 GFLOPS
4000	64	1 x 8	8	hwthread	2.19 s	19.48 GFLOPS
4000	64	2 x 4	8	hwthread	2.00 s	21.35 GFLOPS
4000	64	4 x 2	8	hwthread	1.94 s	22.03 GFLOPS

Table 4: 扩大规模与 8 rank 进程网格结果

最优普通 HPL 参数为 N=4000、NB=64、P x Q=4 x 2、8 个 MPI rank，并使用 --bind-to hwthread --map-by hwthread 绑定。该组合达到 22.03 GFLOPS，相比 4 rank 的 N=4000，P x Q=1 x 4 结果 18.12 GFLOPS 提升约 21.6%。提升不是线性的，原因是 HPL 中面板分解和广播通

信不能随 rank 数完全等比例缩短，同时原始构建使用的参考 BLAS 限制了单 rank 的本地矩阵运算速度。

调优记录还比较了 CPU 绑定策略。在固定 N=2000、NB=64、P x Q=1 x 4、4 rank 时，默认运行约 15.22 GFLOPS，禁用绑定下降到 13.34 GFLOPS，按 core 绑定约 15.17 GFLOPS，按 hwthread 绑定约 16.06 GFLOPS。这说明在短作业中绑定能够减少 rank 迁移带来的缓存扰动，但差距不大，长时间测试仍应重复运行取平均值。

综合调优结果，本实验中最优的普通参考 BLAS 组合为 N=4000，NB=64，P x Q=4 x 2，MPI ranks=8，hwthread binding，性能为 22.03 GFLOPS。

7 Bonus

7.1 更换数学库

本实验完成了数学库更换的 Bonus。以下编译路径为 WSL 主机路径 (/home/cx/workspace/lab1/), HPL 源码编译在 WSL 主机上完成，编译产物通过共享目录分发给容器节点。原始 HPL 构建使用前面从源码编译的参考 CBLAS/BLAS 静态库：

```
LAinc = -I/home/cx/workspace/lab1/CBLAS/include
LAlib = /home/cx/workspace/lab1/CBLAS/lib/cblas_LINUX.a /home/cx/workspace/lab1/
BLAS-3.12.0/blas_LINUX.a -lgfortran -lm
HPL_OPTS = -DHPL_CALL_CBLAS
```

更换后的数学库为 BLIS 0.9.0。BLIS 是面向高性能 BLAS 的实现，提供更好的缓存分块、数据打包和 SIMD 微内核。本实验新建 Make.BLIS，将 HPL 链接到系统 BLIS 动态库：

```
ARCH = BLIS
TOPdir = /home/cx/workspace/lab1/hpl-2.3
MPdir = /usr
MPlib = -L$(MPdir)/lib/x86_64-linux-gnu -lmpi
LAdir = /usr
LAlib = -L/usr/lib/x86_64-linux-gnu -lblis -lm -lpthread
HPL_OPTS = -DHPL_CALL_CBLAS
CC = mpicc
LINKER = mpicc
```

BLIS 软件包和构建配置：

```
(base) cx@LAPTOP-PCMD54F7:~/workspace$ dpkg -l | grep -E 'libblis|blis'
cd /home/cx/workspace/lab1/hpl-2.3
grep -E '^ (ARCH|LAdir|LAinc|LAlib|HPL_OPTS)' Make.BLIS
ii libblis-dev:amd64 0.9.0-1 amd64 BLAS-like Library Instantiation Software Framework (dev,32bit,meta)
ii libblis-pthread-dev:amd64 0.9.0-1 amd64 BLAS-like Library Instantiation Software Framework (dev,32bit,pthread)
ii libblis4:amd64 0.9.0-1 amd64 BLAS-like Library Instantiation Software Framework (32bit,meta)
ii libblis4-openmp:amd64 0.9.0-1 amd64 BLAS-like Library Instantiation Software Framework (32bit,openmp)
ii libblis4-pthread:amd64 0.9.0-1 amd64 BLAS-like Library Instantiation Software Framework (32bit,pthread)
ARCH = BLIS
LAdir = /usr
LAinc = -I/usr/include/x86_64-linux-gnu
LAlib = -L/usr/lib/x86_64-linux-gnu -lblis -lm -lpthread
HPL_OPTS = -DHPL_CALL_CBLAS
ARCHIVER = ar
```

编译 BLIS 版本 HPL，并检查动态链接结果：

```
mpicc -o HPL_pctest.o -c -DHPL_CALL_CBLAS -I/home/cx/workspace/lab1/hpl-2.3/include -I/home/cx/workspace/lab1/hpl-2.3/include/BLIS -I
_pctest.c
mpicc -DHPL_CALL_CBLAS -I/home/cx/workspace/lab1/hpl-2.3/include -I/home/cx/workspace/lab1/hpl-2.3/include/BLIS -I/usr/include/x86_64
/hpl-2.3/bin/BLIS/xhpl HPL_pddriver.o HPL_pdrinfo.o HPL_pctest.o /home/cx/workspace/lab1/hpl-2.3/lib/BLIS/libhpl.a -
ib/x86_64-linux-gnu -lmpi
make /home/cx/workspace/lab1/hpl-2.3/bin/BLIS/HPL.dat
make[3]: Entering directory '/home/cx/workspace/lab1/hpl-2.3/testing/pctest/BLIS'
make[3]: '/home/cx/workspace/lab1/hpl-2.3/bin/BLIS/HPL.dat' is up to date.
make[3]: Leaving directory '/home/cx/workspace/lab1/hpl-2.3/testing/pctest/BLIS'
touch dexe.grd
make[2]: Leaving directory '/home/cx/workspace/lab1/hpl-2.3/testing/pctest/BLIS'
make[1]: Leaving directory '/home/cx/workspace/lab1/hpl-2.3'
libblis.so.4 => /lib/x86_64-linux-gnu/libblis.so.4 (0x00007fce03461000)
```

已有调优记录中，两种版本均使用 $N=4000$ 、 $NB=64$ 、 $P \times Q=4 \times 2$ 、8 个 MPI rank，并固定 $OMP_NUM_THREADS=1$ 、 $BLIS_NUM_THREADS=1$ ，避免 BLAS 线程和 MPI 进程过度并行。以下数据为前期 WSL 主机调优时记录，Make.BLIS 和 bin/BLIS/xhpl 已验证存在（ldd 确认链接 libblis.so.4），但性能数值未在本次检查中重新复测。三次重复运行结果如下：

版本	数学库	运行次数	平均时间	平均性能	相对速度
Original	参考 CBLAS/BLAS	3	2.09 s	20.50 GFLOPS	1.00x
BLIS	BLIS 0.9.0	3	0.73 s	59.05 GFLOPS	2.88x

Table 5: 更换 BLIS 前后的 HPL 性能对比

BLIS 版本的平均性能达到 59.05 GFLOPS，约为原始参考 BLAS 版本 20.50 GFLOPS 的 2.88 倍。残差校验均通过，说明性能提升来自数学库实现差异，而不是错误计算。该结果表明 HPL 的主要耗时集中在 BLAS Level 3 运算中，更换优化 BLAS 是提升 HPL 性能最有效的方式之一。

7.2 使用共享文件系统运行 HPL

除前文使用 NFS 外，额外在现有 Docker 集群中部署了一个最小可用 CephFS，用于验证分布式共享文件系统的基本读写能力。

CephFS 部署在原有 10.42.0.0/24 容器网络中。node01 作为 Ceph 服务端，运行 mon、mgr、mds 和一个基于 2 GiB loopback 文件的 osd.0；node02、node03 作为 CephFS 客户端，通过 ceph-fuse 挂载同一个文件系统到 /mnt/cephfs。该部署是单节点单 OSD 的实验验证环境，不具备高可用能力，但可以说明 CephFS 能够提供跨节点一致的共享目录。

在 node02、node03 上挂载 CephFS 并查看容量：

```
(base) cx@LAPTOP-PCMS4F7:~/workspace/wsl-cluster-lab/cephfs$ docker exec node02 bash /tmp/mount-client.sh
docker exec node03 bash /tmp/mount-client.sh
2026-05-30T04:56:45.887+0000 7f6a617653c0 -1 init, newargv = 0x55754a48e490 newargc=13
2026-05-30T04:56:45.897+0000 7f6a617653c0 -1 init, args.argv = 0x55754a48e660 args argc=4
ceph-fuse[11073]: starting ceph client
ceph-fuse[11073]: starting fuse
Ignoring invalid max threads value 4294967295 > max (100000).
/mnt/cephfs is a mountpoint
Filesystem      Size  Used Avail Use% Mounted on
ceph-fuse       1.9G   0  1.9G   0% /mnt/cephfs
2026-05-30T04:56:46.470+0000 7f6a617653c0 -1 init, newargv = 0x5575b1172280 newargc=13
2026-05-30T04:56:46.470+0000 7f6a617653c0 -1 init, args.argv = 0x5575b1172450 args argc=4
ceph-fuse[10709]: starting ceph client
ceph-fuse[10709]: starting fuse
Ignoring invalid max threads value 4294967295 > max (100000).
/mnt/cephfs is a mountpoint
Filesystem      Size  Used Avail Use% Mounted on
ceph-fuse       1.9G   0  1.9G   0% /mnt/cephfs
```

两个客户端均成功挂载。跨节点读写验证：

```
(base) cx@LAPTOP-PCMD54F7:~/workspace/wsl-cluster-lab/cephfs$ docker exec node02 bash -lc 'mkdir -p /mnt/cephfs/shared-test; printf "writer=node02 time=%s\n" "$(date -Iseconds)" > /mnt/cephfs/shared-test/from-node02.txt; sync; cat /mnt/cephfs/shared-test/from-node02.txt'
docker exec node03 bash -lc 'mkdir -p /mnt/cephfs/shared-test; printf "writer=node03 time=%s\n" "$(date -Iseconds)" > /mnt/cephfs/shared-test/from-node03.txt; sync; cat /mnt/cephfs/shared-test/from-node03.txt'
docker exec node03 bash -lc 'cat /mnt/cephfs/shared-test/from-node02.txt; ls -l /mnt/cephfs/shared-test'
docker exec node02 bash -lc 'cat /mnt/cephfs/shared-test/from-node03.txt; ls -l /mnt/cephfs/shared-test'
writer=node02 time=2026-05-30T04:57:57+00:00
writer=node03 time=2026-05-30T04:57:58+00:00
writer=node02 time=2026-05-30T04:57:57+00:00
total 1
-rw-r--r-- 1 root root 45 May 30 04:57 from-node02.txt
-rw-r--r-- 1 root root 45 May 30 04:57 from-node03.txt
writer=node03 time=2026-05-30T04:57:58+00:00
total 1
-rw-r--r-- 1 root root 45 May 30 04:57 from-node02.txt
-rw-r--r-- 1 root root 45 May 30 04:57 from-node03.txt
```

node02 写入的 from-node02.txt 可以在 node03 读取，node03 写入的 from-node03.txt 也可以在 node02 读取，说明两个客户端访问的是同一个 CephFS 文件系统。

Ceph 集群状态：

```
(base) cx@LAPTOP-PCMD54F7:~/workspace/wsl-cluster-lab/cephfs$ docker exec node01 ceph -s
docker exec node01 ceph fs status cephfs
cluster:
  id: eb2977b6-65c1-44dc-855a-9a6f42999f35
  health: HEALTH_WARN
    mon is allowing insecure global_id reclaim
    Module 'volumes' has failed dependency: No module named 'distutils'
    1 monitors have not enabled msgr2
    3 pool(s) have no replicas configured

services:
  mon: 1 daemons, quorum node01 (age 47m)
  mgr: node01(active, since 45m)
  mds: 1/1 daemons up
  osd: 1 osds: 1 up (since 44m), 1 in (since 45m)

data:
  volumes: 1/1 healthy
  pools: 3 pools, 41 pgs
  objects: 27 objects, 495 KiB
  usage: 27 MiB used, 2.0 GiB / 2.0 GiB avail
  pgs: 41 active+clean

cephfs - 2 clients
=====
RANK STATE MDS ACTIVITY DNS INOS DIRS CAPS
0 active node01 Reqs: 0 /s 13 16 13 8
POOL TYPE USED AVAIL
cephfs_metadata metadata 80.0k 1914M
cephfs_data data 8192 1914M
MDS version: ceph version 19.2.0~git20240301.4c76c50 (4c76c50a73f63ba48ccdf0adccce03b00d1d80c7) squid (dev)
```

7.3 K3s 基础部署

K3s 部署在同一组 Docker 节点上，其中 node01 运行 k3s server，node02、node03、node04 作为 k3s agent 加入集群。由于 WSL/Docker-in-Docker 场景中容器内 overlayfs 不可用，启动 k3s 时使用 `--snapshotter native`；同时容器需要以 `--privileged` 和 `--cgroupns=host` 方式运行，才能满足 kubelet 对 cgroup 和网络能力的要求。

检查部署脚本和关键启动参数：

```
(base) cx@LAPTOP-PCMD54F7:~/workspace/wsl-cluster-lab/cephfs$ cd /home/cx/workspace/wsl-cluster-lab
grep -nE 'privileged|cgroupns|snapshotter|k3s server|k3s agent|cluster-cidr|pause-image' setup.sh k3s-setup.sh
setup.sh:77:         --privileged \
setup.sh:78:         --cgroupns=host \
setup.sh:90:         --privileged \
setup.sh:91:         --cgroupns=host \
k3s-setup.sh:75:         echo "${node} must run with Docker privileged=true and cgroupns=host; got: ${flags}" >&2
k3s-setup.sh:112:     nohup k3s server \
k3s-setup.sh:116:         --cluster-cidr 10.244.0.0/16 \
k3s-setup.sh:117:         --pause-image ${PAUSE_IMAGE} \
k3s-setup.sh:118:         --snapshotter native \
k3s-setup.sh:145:     nohup k3s agent \
k3s-setup.sh:151:         --pause-image ${PAUSE_IMAGE} \
k3s-setup.sh:152:         --snapshotter native \
```

k3s-setup.sh 中 server 的核心启动参数如下：

```
k3s server \
--node-ip 10.42.0.11 \
--node-external-ip 10.42.0.11 \
--flannel-iface eth0 \
--cluster-cidr 10.244.0.0/16 \
--pause-image local-pause:3.6 \
--snapshotter native \
--write-kubeconfig-mode 0644 \
--disable traefik
```

agent 节点使用 node01 生成的 token 加入集群，关键参数如下：

```
k3s agent \
--server https://node01:6443 \
--token <node-token> \
--node-ip <node-ip> \
--node-external-ip <node-ip> \
--flannel-iface eth0 \
--pause-image local-pause:3.6 \
--snapshotter native
```

部署和验证 K3s:

```
== Hostname and IP resolution ==
10.42.0.11      node01
10.42.0.12      node02
10.42.0.13      node03
10.42.0.14      node04
10.42.0.11      node01
10.42.0.12      node02
10.42.0.13      node03
10.42.0.14      node04
10.42.0.11      node01
10.42.0.12      node02
10.42.0.13      node03
10.42.0.14      node04
10.42.0.11      node01
10.42.0.12      node02
10.42.0.13      node03
10.42.0.14      node04
== UID/GID consistency ==
uid=2001(labuser) gid=2001(labuser) groups=2001(labuser)
uid=2001(labuser) gid=2001(labuser) groups=2001(labuser)
uid=2001(labuser) gid=2001(labuser) groups=2001(labuser)
uid=2001(labuser) gid=2001(labuser) groups=2001(labuser)
== Time drift check ==
node01 drift=0s
node02 drift=0s
node03 drift=0s
node04 drift=0s
== SSH passwordless login from node01 ==
node02
node03
node04
```

```

== Slurm configuration consistency ==
badb8a96265b1d36b70e32abf4b56921e88e8cb48317b33dc61aa73fd6179a90 /etc/slurm/slurm.conf
badb8a96265b1d36b70e32abf4b56921e88e8cb48317b33dc61aa73fd6179a90 /etc/slurm/slurm.conf
badb8a96265b1d36b70e32abf4b56921e88e8cb48317b33dc61aa73fd6179a90 /etc/slurm/slurm.conf
badb8a96265b1d36b70e32abf4b56921e88e8cb48317b33dc61aa73fd6179a90 /etc/slurm/slurm.conf
== Slurm service validation ==
44
26
27
27
Sat May 30 05:06:24 2026
NODELIST    NODES PARTITION      STATE CPUS      S:C:T MEMORY  TMP_DISK  WEIGHT  AVAIL_FE  REASON
node02         1   debug*        idle  1        1:1:1   512        0        1    (null) none
node03         1   debug*        idle  1        1:1:1   512        0        1    (null) none
node04         1   debug*        idle  1        1:1:1   512        0        1    (null) none
== Slurm srun validation ==
node02
== Slurm sbatch NFS output validation ==
Submitted batch job 2
node02
/
slurm nfs output ok
node02
/
slurm nfs output ok
OK: SSH, NFS, MUNGE, sinfo, srun, sbatch, and Slurm NFS output checks passed.
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
  0     0     0     0     0     0      0      0  --:--:--  --:--:--  --:--:--     0
37 62.2M   37 23.4M    0     0 2604k      0  0:00:24  0:00:09  0:00:15 3175k

```

查看节点、Pod 和 Service:

```

(base) cx@LAPTOP-PCMD54F7:~/workspace$ sudo docker exec node01 kubectl get nodes -o wide
sudo docker exec node01 kubectl get pods -o wide
sudo docker exec node01 kubectl get service k3s-demo -o wide
[sudo] password for cx:
NAME          STATUS    ROLES          AGE   VERSION          INTERNAL-IP    EXTERNAL-IP    OS-IMAGE          KERNEL-VERSION
CONTAINER-RUNTIME
node01        Ready    control-plane,master  62s   v1.28.15+k3s1    10.42.0.11     10.42.0.11     Ubuntu 24.04 LTS  5.15.167.4-micr
osoft-standard-WSL2   containerd://1.7.22-k3s1.28
node02        Ready    <none>          60s   v1.28.15+k3s1    10.42.0.12     10.42.0.12     Ubuntu 24.04 LTS  5.15.167.4-micr
osoft-standard-WSL2   containerd://1.7.22-k3s1.28
node03        Ready    <none>          59s   v1.28.15+k3s1    10.42.0.13     10.42.0.13     Ubuntu 24.04 LTS  5.15.167.4-micr
osoft-standard-WSL2   containerd://1.7.22-k3s1.28

```

kubectl get nodes -o wide 显示 node01、node02、node03 三个节点均为 Ready。

通过 NodePort 访问测试应用:

```

(base) cx@LAPTOP-PCMD54F7:~/workspace$ sudo docker exec node01 curl -fsS http://10.42.0.11:30080/
[sudo] password for cx:
<!doctype html><title>k3s demo</title><h1>k3s demo ok</h1>

```

8 AI Agent 使用说明

由于时间有限，我使用 gpt + codex (plus 套餐) 辅助完成实验报告。gpt 用来解答知识性问题和安装文件，codex 用来做 bonus，debug，生成验证脚本以及撰写实验报告。

codex 使用 docker 完成大部分实验，依本人的水平出错也无法核验。但 AI Agent 的特性是，只要目标明确，token 充足，就可以完成任务。在完成 lab1 的过程中，笔者的 codex 在 K3s 部署部分遇到挑战，花了相当久的时间才部署完成（也和网络不稳定有关）。最近实验压力较大。暑假，为了更好地理解 HPC 以及了解基本的 coding 命令，会适当减少 AI Agent 的暴力使用。

9 附录

9.1 Slurm 作业脚本

本实验中主要使用两个 Slurm 作业脚本。第一个脚本用于验证 Slurm 能够完成基本调度和执行，文件路径为 `/cluster/shared/slurm-test.sbatch`：

```
#!/bin/bash
#SBATCH --job-name=slurm-test
#SBATCH --partition=debug
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --output=/cluster/shared/slurm-test-%j.out

hostname
pwd
srun hostname
```

第二个脚本用于通过 Slurm 启动 HPL，文件路径为 `/cluster/shared/hpl/run-hpl.sbatch`：

```
#!/bin/bash
#SBATCH --job-name=hpl
#SBATCH --partition=debug
#SBATCH --nodes=3
#SBATCH --ntasks-per-node=1
#SBATCH --output=/cluster/shared/hpl/hpl-%j.out
#SBATCH --error=/cluster/shared/hpl/hpl-%j.err

set -euo pipefail
cd /cluster/shared/hpl
srun --mpi=pmix -N3 --ntasks=3 ./xhpl
```

提交作业时使用 `sbatch --parsable` 获取作业号，再按作业号读取输出文件，避免固定写死历史 job id：

```
jobid=$(sbatch --parsable /cluster/shared/hpl/run-hpl.sbatch)
echo "Submitted batch job ${jobid}"
squeue -j "${jobid}"
cat /cluster/shared/hpl/hpl-${jobid}.out
```

9.2 HPL 输出摘要

```
N = 120
NB = 32
P x Q = 1 x 3
MPI ranks = 3
Output file = /cluster/shared/hpl/hpl-4.out
```

T/V	N	NB	P	Q	Time	Gflops
-----	---	----	---	---	------	--------


```

WR00R2R2      120    32    1    3      0.01      1.2523e-01
||Ax-b||_oo/(eps*(||A||_oo*||x||_oo+||b||_oo)*N)= 1.34012681e-02 ..... PASSED
End of Tests.

```

WSL 本地参数调优中的最优普通参考 BLAS 结果为：

```

N = 4000
NB = 64
P x Q = 4 x 2
MPI ranks = 8
MPI binding = --bind-to hwthread --map-by hwthread
Observed performance = 22.03 GFLOPS
Residual check = PASSED

```

更换 BLIS 数学库后的对比结果为：

```

Original reference BLAS:
Average time = 2.09 s
Average performance = 20.50 GFLOPS
Relative speed = 1.00x

BLIS 0.9.0:
Average time = 0.73 s
Average performance = 59.05 GFLOPS
Relative speed = 2.88x

```

9.3 修改文件说明

本实验主要修改和新增了以下文件：

文件	修改内容	原因
/etc/hosts	加入 node01 至 node04 的静态解析	保证 MPI、NFS、Slurm、CephFS 和 K3s 可通过节点名通信
/etc/exports	导出 /cluster/shared 给 10.42.0.0/24	提供 NFS 共享目录
/etc/slurm/slurm.conf	配置 node01 为控制节点, node02 至 node04 为计算节点	启用 Slurm debug 分区
BLAS-3.12.0/make.inc	设置 FORTRAN=gfortran、FFLAGS=-O3 -fallow-argument-mismatch	从源码编译参考 BLAS 静态库
CBLAS/Makefile.in	设置 BLIB 指向 blas_LINUX.a、FC=gfortran	从源码编译 CBLAS 静态库
hpl-2.3/Make.Linux	设置 LAlib 链接 CBLAS/BLAS 静态库、CC=mpicc	从源码编译 HPL (参考 BLAS 版本)
hpl-2.3/Make.BLIS	设置 LAlib=-lblis 动态链接 BLIS	Bonus: 更换数学库为 BLIS
/cluster/shared/slurm-test.sbatch	新增 Slurm 调度验证脚本	验证作业提交和 srun 执行
/cluster/shared/hpl/run-hpl.sbatch	新增 HPL 批处理脚本	通过 Slurm 启动 HPL
/cluster/shared/hpl/HPL.dat	设置 N=120, NB=32, P=1, Q=3	匹配 3 个 MPI rank 的集群验证运行

Table 6: 基础配置与 Slurm/HPL 文件

文件	修改内容	原因
/home/cx/workspace/lab1/hpl-2.3/Make.BLIS	新增 BLIS 链接配置	对比参考 BLAS 与 BLIS 的 HPL 性能
wsl-cluster-lab/cephfs/bootstrap-node01.sh	创建 Ceph mon/mgr/mds/osd 和 CephFS	验证 CephFS 共享文件系统
wsl-cluster-lab/cephfs/mount-client.sh	使用 ceph-fuse 挂载 /mnt/cephfs	验证客户端挂载和跨节点读写
wsl-cluster-lab/k3s-setup.sh	启动 k3s server/agent 并使用 --snapshotter native	适配 WSL/Docker-in-Docker 环境
wsl-cluster-lab/k3s-verify.sh	创建 k3s-demo Deployment 和 NodePort Service	验证 Pod 调度和服务访问

Table 7: Bonus 部分脚本与构建文件

其中, HPL 更换数学库时没有修改 HPL 源码, 只新增了 Make.BLIS 并重新编译生成 bin/BLIS/xhpl。CephFS 和 K3s 部分均为实验环境脚本, 用于在 Docker 容器节点中复现实验配置。